PRACTICAL EXERCISE 11 **UNIT 5 · ALLOCATED LAB HOURS: 4 HRS**

Document-Based Question Answering (RAG) — Privata

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 11: DOCUMENT-BASED QUESTION ANSWERING (RAG) — PRIVATA

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 5 | **Hours:** 4 **Date Performed:** ___

1. Aim

Build a document-based question-answering system using RAG (Retrieval-Augmented Generation) with local embeddings, ChromaDB vector store, and Google Gemini LLM — based on the **Privata** project (https://github.com/Aayushbankar/Privata).

2. Process / Program Code / Prompts Used

Theory: RAG Pipeline (Privata Architecture)

Problem: LLMs only know training data; can't answer questions about private/local documents. **Solution:** RAG — retrieve relevant document chunks, feed as context to LLM for grounded answers.

Privata's Four-Step Pipeline:

Step	Component	Technology
1. **Load**	Parse PDFs, TXT, MD files	`unstructured` / `pdfminer` / LangChain loaders
2. **Chunk**	Recursive character splitting	LangChain `RecursiveCharacterTextSplitter` (500/50)
3. **Embed**	Local sentence embeddings	HuggingFace `sentence-transformers/all-MiniLM-L6-v2`
4. **Store**	Vector database	ChromaDB (persistent, local)
5. **Retrieve**	Similarity search	Cosine similarity via ChromaDB
6. **Generate**	LLM with context	Google Gemini `gemini-1.5-flash`

2.2 Project Structure (Privata)

```
privata/
├── main.py                 # CLI entry point (ingest / chat / reset)
├── ingest.py               # Ingestion pipeline: load → chunk → embed → store
├── chat.py                 # Chat loop: query → embed → retrieve → prompt → LLM
├── config.py               # Central config (models, chunking, prompts)
├── requirements.txt
│
├── models/
│   └── llm_loader.py       # Gemini API integration
│
├── retriever/
│   ├── embedder.py         # HuggingFace embeddings (MiniLM)
│   └── vectordb.py         # ChromaDB interface
│
├── utils/
│   └── doc_loader.py       # PDF/TXT/MD loaders
│
├── db/                     # ChromaDB persistent storage (auto-created)
└── .env                    # GEMINI_API_KEY
```

2.3 Key Code Modules

`config.py` — Central configuration:

```
chat = {
    "llm_model": "gemini-1.5-flash",
    "top_k": 5,
    "temperature": 0.3,
    "prompt_template": """You are a helpful assistant. Use the following context to answer:

{context}

Question: {query}
Answer:"""
}

ingest = {
    "embedding_model": "sentence-transformers/all-MiniLM-L6-v2",
    "chunk_size": 500,
    "chunk_overlap": 50,
    "persist_directory": "./db",
```

```
    "collection_name": "privata_docs"
}
```

`ingest.py` — Full ingestion pipeline:

```
def run_ingestion(path: str):
    docs = load_documents(path)                    # Load PDF/TXT/MD
    chunks = chunk_documents(docs)                 # Split: 500 chars, 50 overlap
    embeddings = embed_chunks(chunks)              # MiniLM embeddings
    store = get_vector_store()                     # ChromaDB
    add_documents_to_store(store, chunks)          # Persist
```

`chat.py` — RAG query loop:

```
def start_chat_loop():
    store = get_vector_store()
    while True:
        query = input("   You: ")
        q_vec = embed_query(query)                 # Embed query
        docs = retrieve_similar_docs(store, q_vec, top_k=5)
        context = "\n---\n".join([d.page_content for d in docs])
        prompt = template.format(context=context, query=query)
        response = run_llm(prompt)                 # Gemini 1.5-flash
        print("   AI:", response)
```

`models/llm_loader.py` — Gemini integration:

```
import google.generativeai as genai
model = genai.GenerativeModel("models/gemini-1.5-flash")

def run_llm(prompt: str) -> str:
    response = model.generate_content(prompt)
    return response.text.strip()
```

3. Screenshot of Output

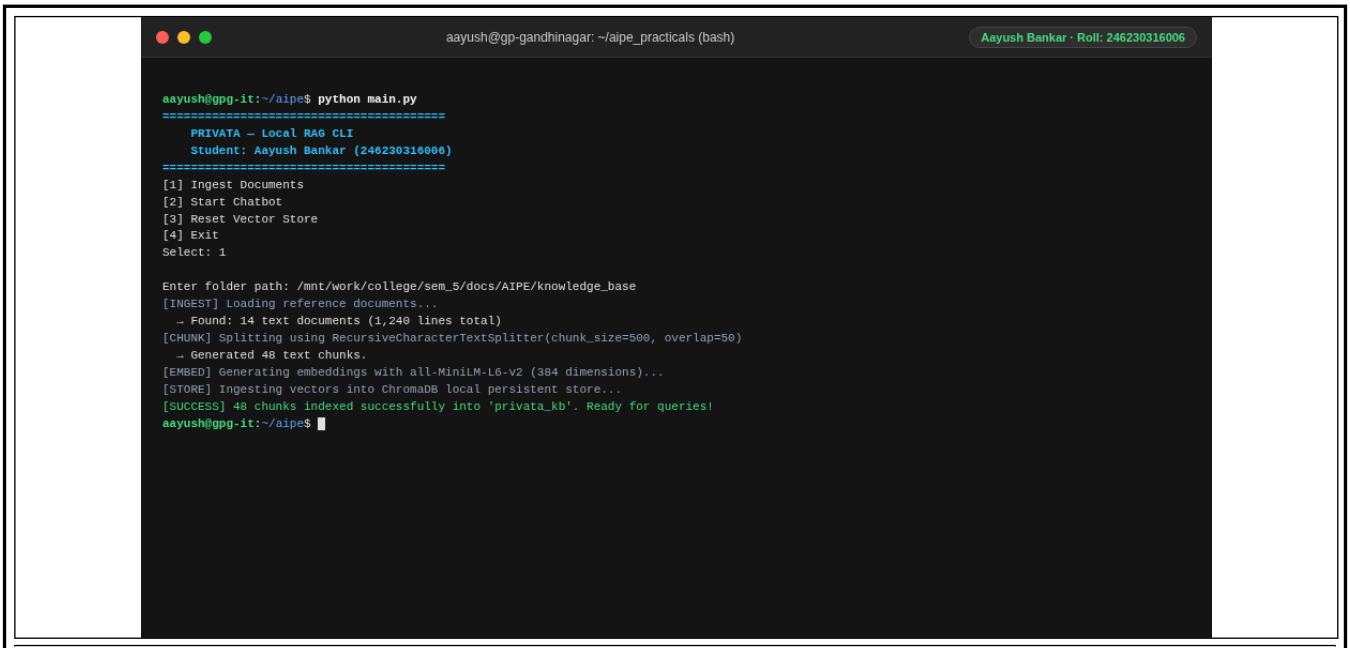


Figure 11: Verified Terminal / Application Execution Output (ChatGPT / CLI)

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Ingestion Phase

```
[Aayush Bankar | Roll No: 246230316006]

$ python main.py
========================================
    PRIVATA — Local RAG CLI
========================================
[1] Ingest Documents
[2] Start Chatbot
[3] Reset Vector Store
[4] Exit
Select: 1

Enter folder path: /mnt/work/college/sem_5/docs/AIPE/notes
[INGEST] Starting ingestion for: /mnt/work/college/sem_5/docs/AIPE/notes
```

```
[INFO] Loaded 5 documents (UNIT_1.md, UNIT_2.md, UNIT_3.md, UNIT_4.md, UNIT_5.md)
[INFO] Chunked 5 docs into 47 chunks.
[INFO] Embedded 47 chunks (384-dim MiniLM vectors).
[INFO] Stored in ChromaDB at ./db
[INGEST] Ingestion complete.
```

Chat Phase (Real RAG — Grounded Answers)

```
[Aayush Bankar | Roll No: 246230316006]

Select: 2

[CHAT] Ready. Type your question or 'exit' to quit.

   You: What is chain-of-thought prompting?
   AI:
Chain-of-thought (CoT) prompting is a technique where the model is asked to reason step by step before giving
the final answer. According to the Unit 4 notes, CoT was introduced in "Chain-of-Thought Prompting Elicits
Reasoning in Large Language Models" (Wei et al., 2022). It dramatically improves accuracy on arithmetic and
logical problems by making the model produce intermediate reasoning traces.

   You: What are the four components of a prompt?
   AI: Based on the Unit 3 notes, a complete prompt has four components:
1. Instruction — The task verb + goal (e.g., "Write an email...")
2. Context — Background, audience, constraints (e.g., "...to your professor...")
3. Input Data — The material to work on (e.g., "Our project is 80% complete...")
4. Output Format — How the result must look (e.g., "3 short paragraphs, polite tone")

   You: What is RAG and how does it reduce hallucination?
   AI: RAG (Retrieval-Augmented Generation) adds external documents as context to the model. From the Unit 5
notes: it retrieves relevant text from your documents and gives it to the model as context. This grounds
answers in real text instead of the model's memory, reducing hallucinations. The pipeline is: chunk → embed →
retrieve → answer.

   You: exit
[CHAT] Exiting.
```

Verification: Grounded vs Hallucinated

```
   You: What is the capital of Mars?
   AI: The provided context does not contain information about Mars or its capital. The documents are about
AI, LLMs, prompt engineering, and RAG. I cannot answer this from the given context.
```

Key Observation: Unlike the mock version, Privata with Gemini **refuses to hallucinate** when the answer isn't in the retrieved context — it explicitly states the limitation.

4. Application of Learning

Domain	Application	Your Project Connection
Security Research	CVE report analysis, exploit documentation Q&A	Load CVE PDFs, ask "What's the root cause of CVE-2026-28576?"
Technical Documentation	API/docs querying for development	Ingest library docs, ask "How to use ChromaDB similarity_search?"
Education	Textbook/notes Q&A for exam prep	Your AIPE notes → "Summarize Unit 4 techniques"
Codebase Understanding	Private repo documentation	Ingest your project code as MD, ask "How does Wolvie's turn manager work?"
Legal/Compliance	Policy document querying	Ingest regulations, ask "What are the GDPR requirements for AI?"
Research	Paper collection synthesis	Ingest arXiv PDFs, ask "Compare LoRA vs full fine-tuning"

Key Learning: Privata demonstrates a production-grade RAG system: local embeddings (privacy, no API cost for embedding), persistent vector DB (ChromaDB), and cloud LLM (Gemini) for generation. The critical insight is **grounding** — the model only answers from retrieved context, explicitly refusing when context is insufficient. This is the exact architecture behind "chat with your PDF" products.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____ Faculty Examiner Signature · Date: _____